

Relatório – Controle de Fluxo com BRAM e FIFO

Disciplina: Lógica Reconfigurável – Engenharia de Computação (UTFPR)

Aluno: Felipe Costa Santos

1. Objetivo

Implementar e validar um sistema de controle de fluxo entre dois blocos de memória utilizando uma FIFO como buffer intermediário. O sistema deve atender aos seguintes requisitos:

- BRAM de origem preenchida com valores crescentes de 0 a 2047;
- Transferência dos dados da BRAM de origem para a BRAM de destino;
- Escrita na FIFO ocorrendo a uma taxa **7 vezes maior** que a leitura;
- Controle de fluxo baseado na ocupação da FIFO:
- pausa da escrita em 75% da capacidade;
- retomada em 50%.

2. Procedimentos adotados

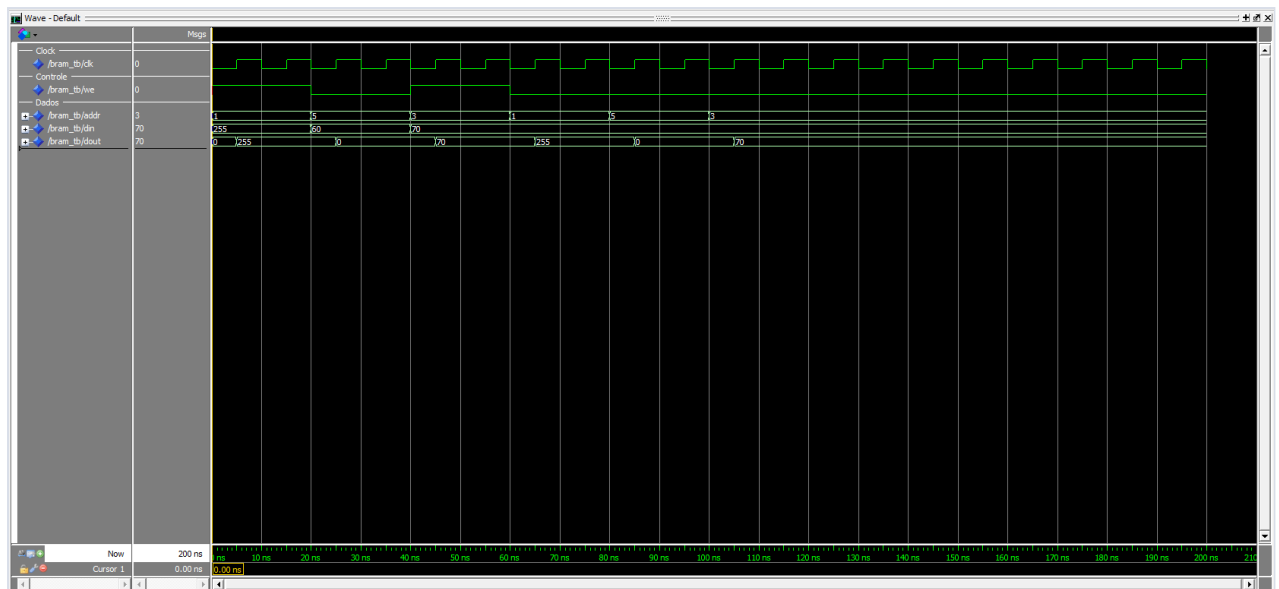
2.1 Validação dos blocos individuais

Os blocos individuais foram gerados no Quartus e testados separadamente antes da integração. Para manter a interface do restante do projeto, cada IP foi encapsulada em um wrapper VHDL simples.

- **BRAM (2048 x 8 bits)**

Implementada com a megafunction `altsyncram` gerada pelo wizard do Quartus. Foram usadas duas variantes do mesmo bloco: uma inicializada por arquivo `.mif` para a BRAM de origem e outra sem inicialização para a BRAM de destino. A simulação confirmou o correto acesso aos dados e o comportamento esperado da memória gerada.

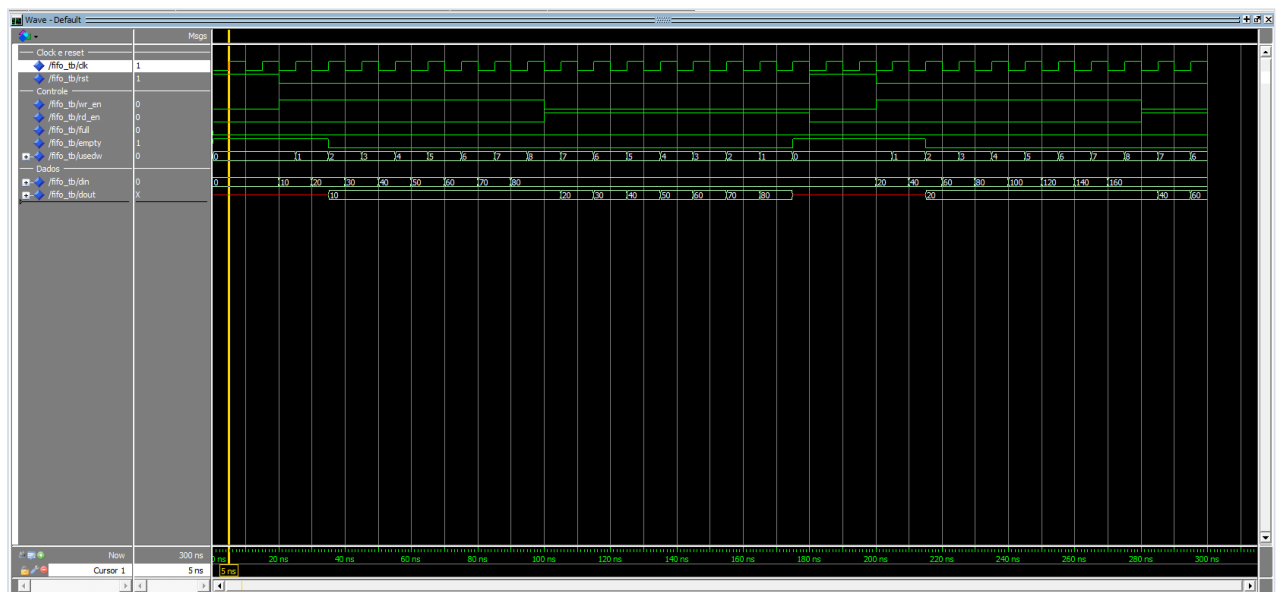
- **Simulação da BRAM:**



- **FIFO (1024 x 8 bits)**

Implementada com a megafunção `scfifo` gerada pelo wizard do Quartus. Foram verificados o funcionamento dos sinais `full`, `empty` e `usedw`, além da preservação da ordem dos dados na fila.

- **Simulação da FIFO:**



2.2 Integração do sistema completo

Os blocos foram integrados formando o seguinte fluxo:

BRAM origem → FIFO → BRAM destino

O controlador implementado foi responsável por:

- Gerar endereços de leitura e escrita;
- Controlar `wr_en` e `rd_en`;
- Monitorar a ocupação da FIFO (`usedw`);

- Garantir a razão de velocidade 7:1;
- Sinalizar o término da operação (`done`).

A leitura foi desacelerada por um contador, permitindo uma operação a cada 7 ciclos de clock.

3. Detalhes de implementação

3.1 BRAM

- Bloco gerado pelo Quartus a partir da megafunction `altsyncram`, em modo de porta simples;
- Capacidade de 2048 palavras de 8 bits;
- O wrapper `sync_bram` mantém a interface do projeto e seleciona a variante da IP por meio do `generic INIT_SEQUENCE`;
- Na BRAM de origem é usada a versão `bram_gen_Initialized`, carregada a partir do arquivo `bram_init_data.mif`;
- Na BRAM de destino é usada a versão `bram_gen`, sem arquivo de inicialização.

3.2 FIFO

- Bloco gerado pelo Quartus a partir da megafunction `scfifo`;
- Capacidade de 1024 palavras de 8 bits;
- Sinais `full`, `empty` e `usedw` fornecidos diretamente pela própria IP;
- Reset conectado ao pino `aclr` da FIFO gerada;
- O wrapper `sync_fifo` apenas adapta a interface do projeto e converte `usedw` para o tipo `unsigned` usado pelo controlador.

3.3 Controlador de fluxo

Duas máquinas de estado foram utilizadas, que operam de forma paralela e coordenada para controlar a escrita e leitura:

Escrita

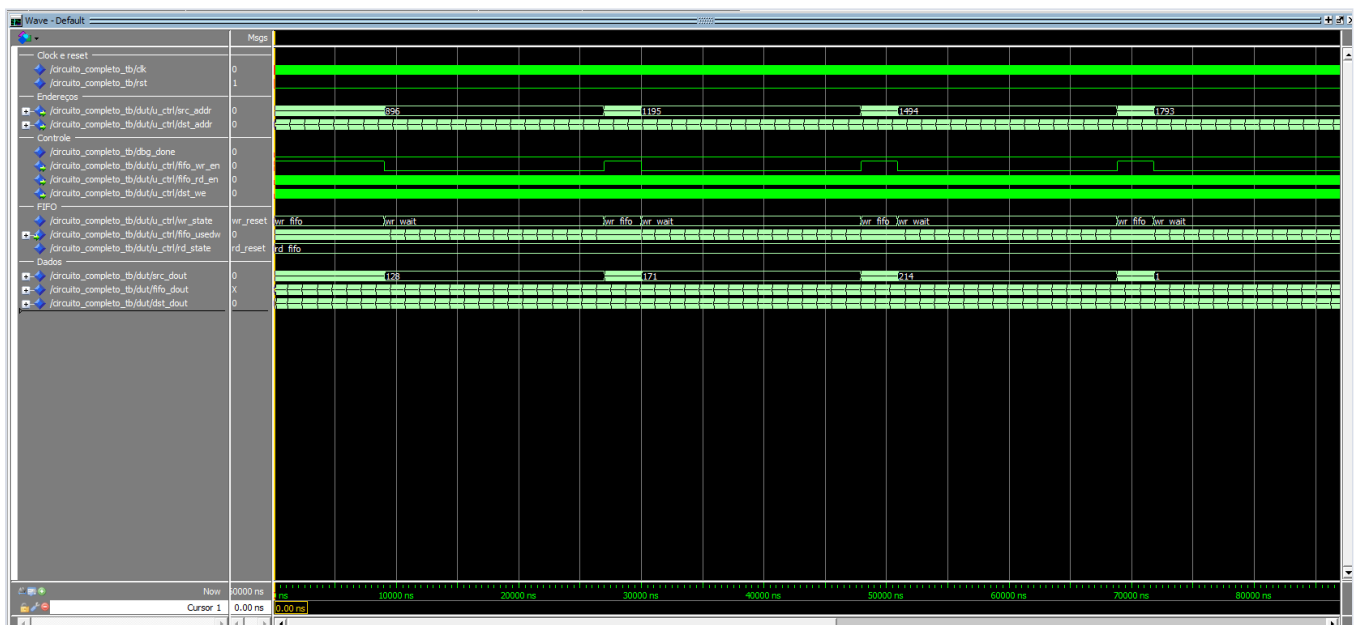
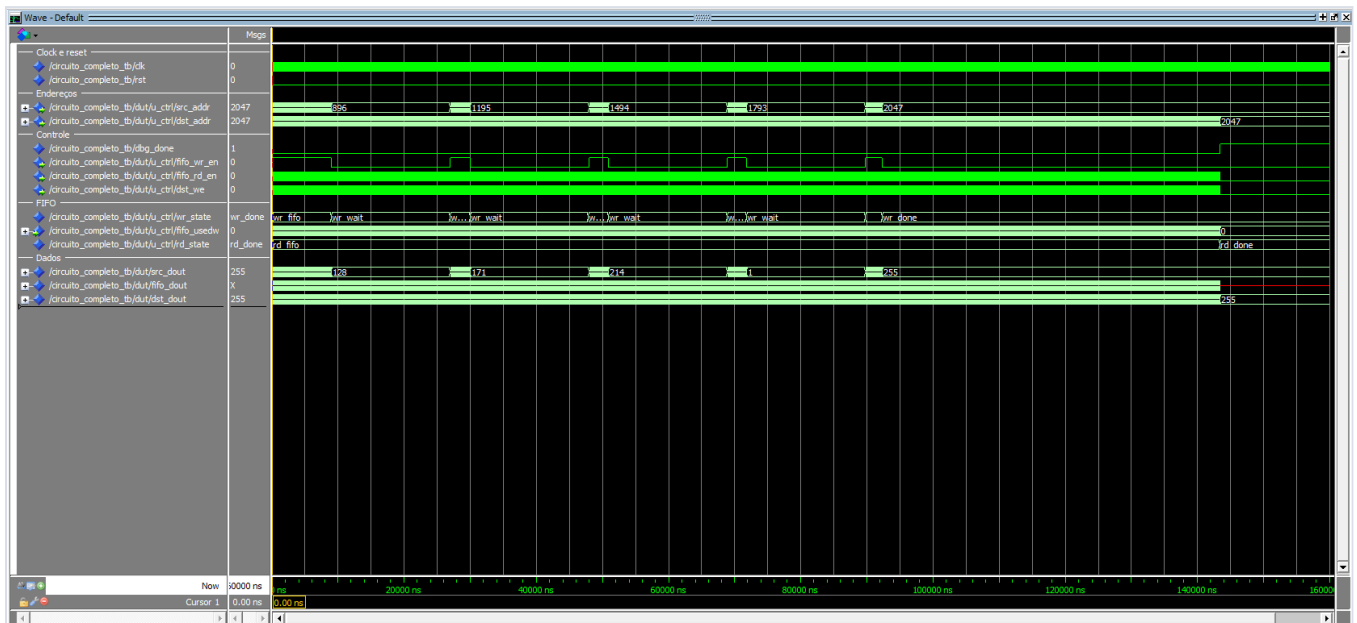
- `wr_reset` → `wr_load` → `wr_fifo` → `wr_wait` → `wr_fifo` → `wr_wait` → ... → `wr_done`
- Pausa quando `usedw` ≥ 75%;
- Retoma quando `usedw` ≤ 50%.

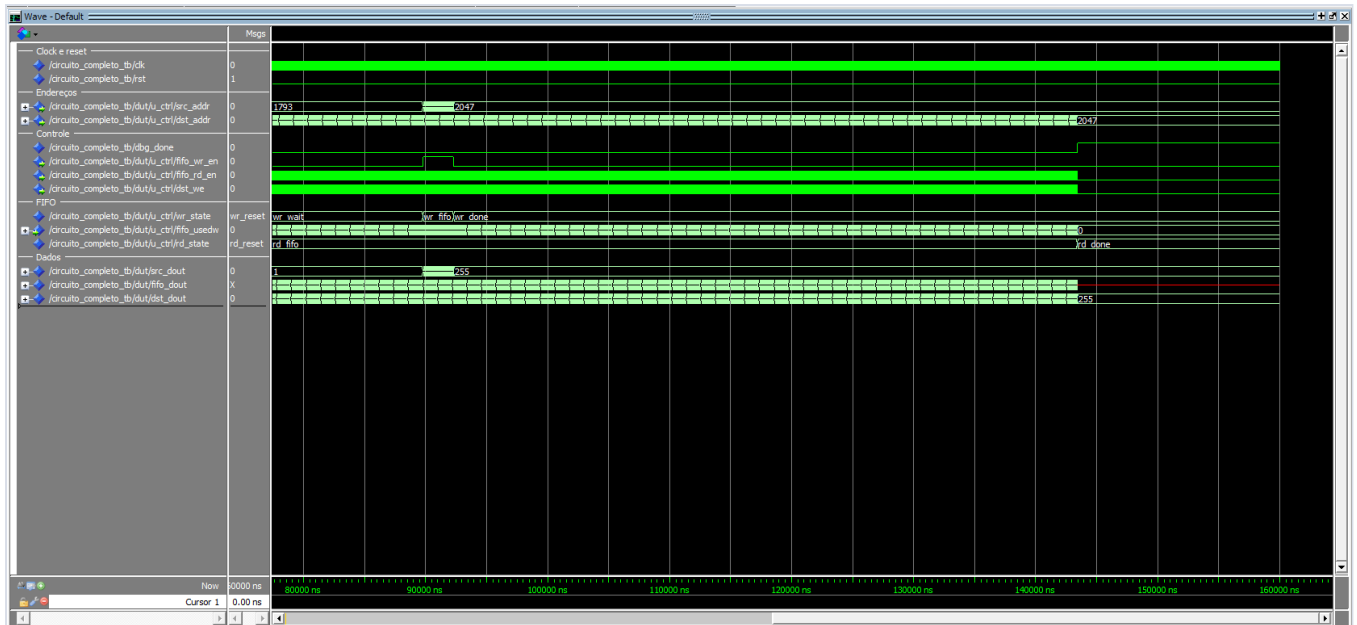
Leitura

- `rd_reset` → `rd_wait` → `rd_fifo` → `rd_done`
- Leitura condicionada à presença de dados;
- Frequência reduzida (1 leitura a cada 7 clocks).

4. Resultados da simula?o

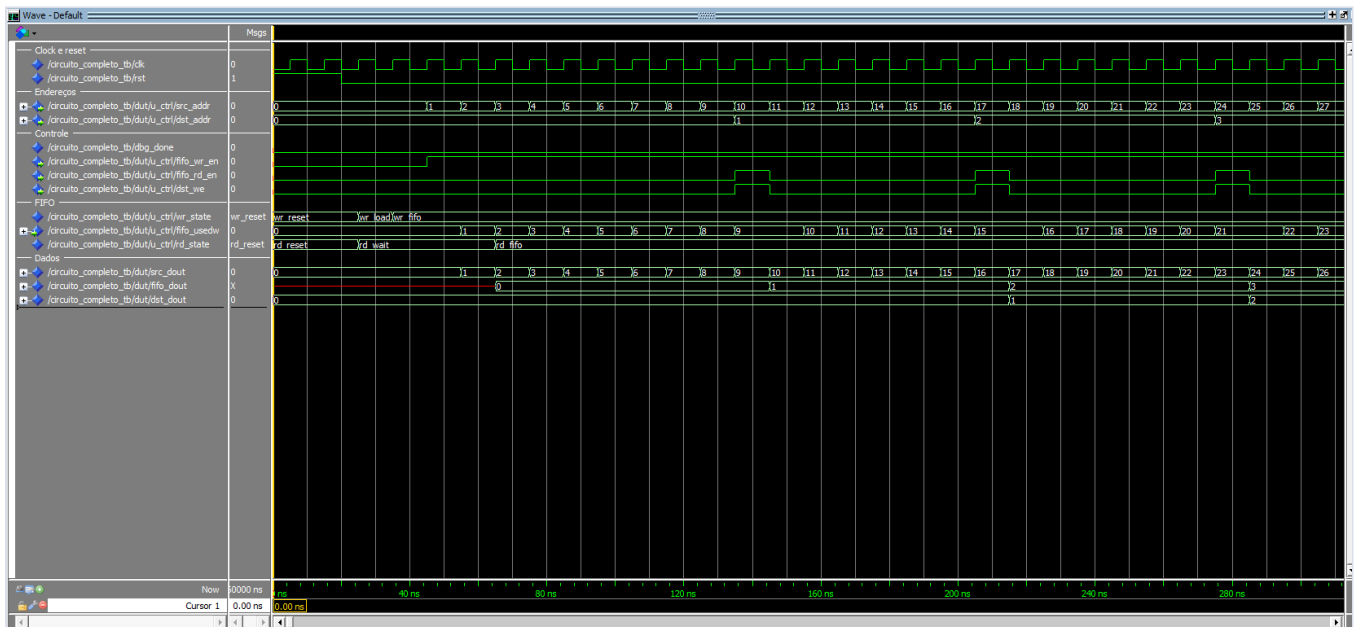
4.1 Vis?o geral da simula?o





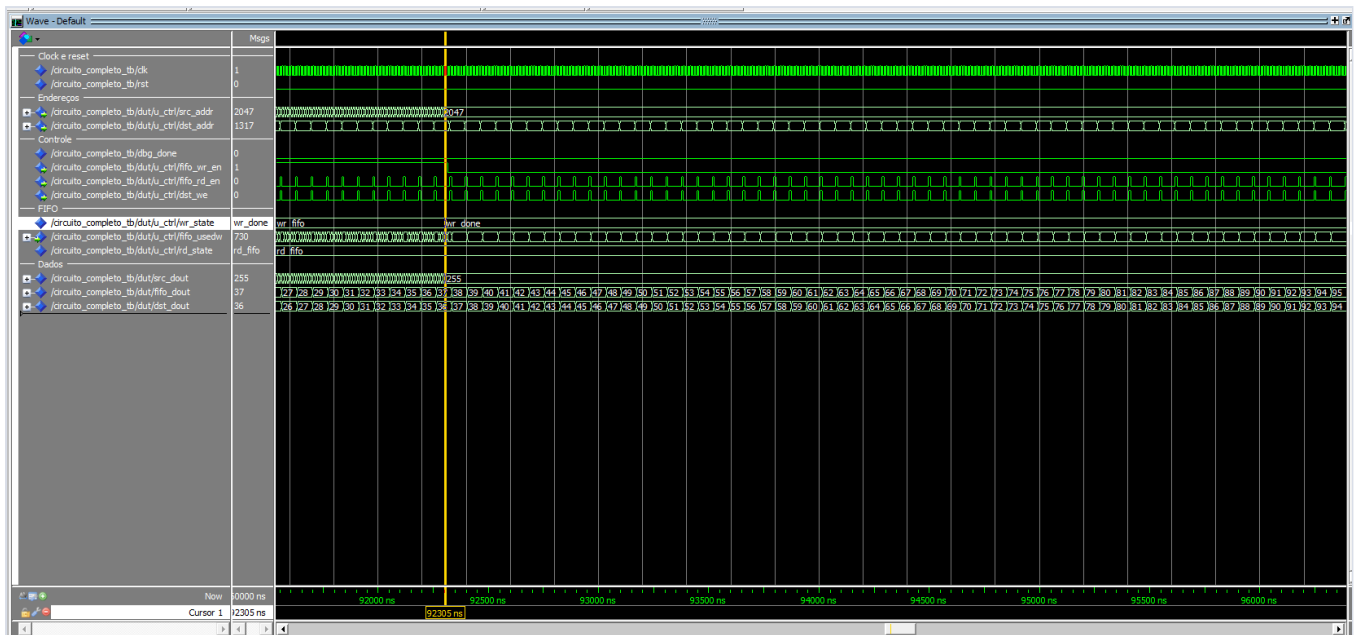
A primeira imagem resume toda a transferência em uma única visão. As outras duas apenas mostram a simulação com um pouco mais de zoom, principalmente para destacar os estados `wr_fifo` intermediários. Juntas, as três imagens mostram os endereços de origem e destino chegando ao último endereço, a alternância entre `wr_fifo` e `wr_wait`, a leitura acontecendo ao longo do processo e o encerramento com `wr_done` e `rd_done`. Essa é a melhor captura para entender o ciclo inteiro sem precisar analisar os detalhes finos.

4.2 Início da operação



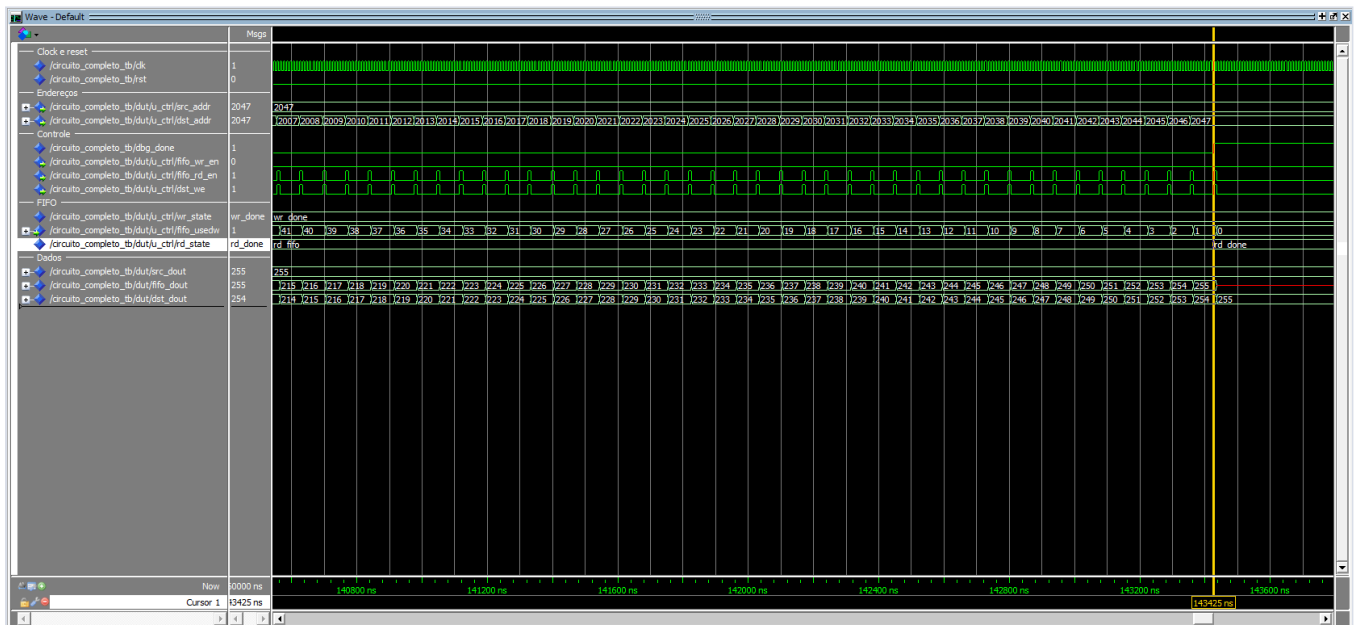
Aqui aparece o começo da operação logo após a liberação do reset. Dá para ver o controlador saindo de `wr_reset`, passando por `wr_load` e começando a alimentar a FIFO quando entra em `wr_fifo`. Ao mesmo tempo, a leitura sai de `rd_reset` para `rd_wait`, esperando a FIFO ter dados, e depois vai para `rd_fifo` no primeiro clock em que a FIFO já contém dados. Essa imagem é útil para entender o início do sistema.

4.3 Término da escrita



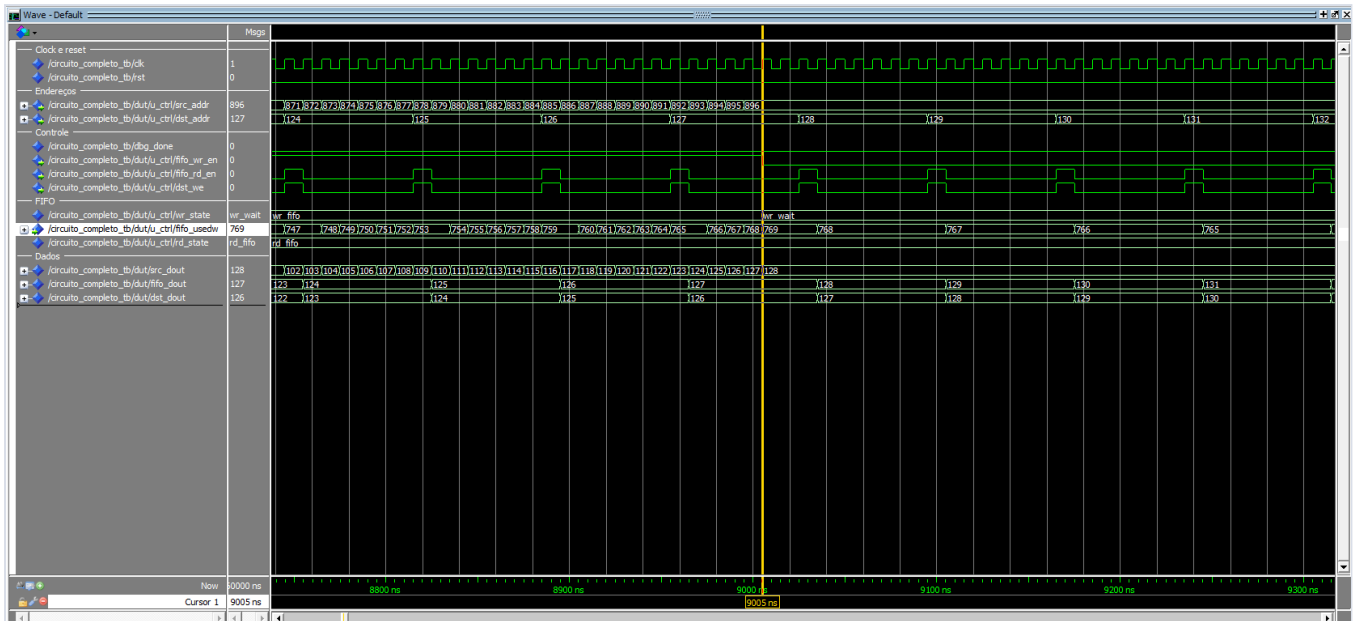
Essa captura marca o instante em que a BRAM de origem já chegou ao último endereço e a máquina de escrita passou para `wr_done`. Mesmo com a escrita encerrada, a FIFO ainda contém dados e a máquina de leitura continua em `rd_fifo`. Isso confirma que o sistema não para no momento da última escrita; ele segue até drenar a fila e transferir todos os dados da primeira BRAM para a segunda.

4.4 Término da leitura



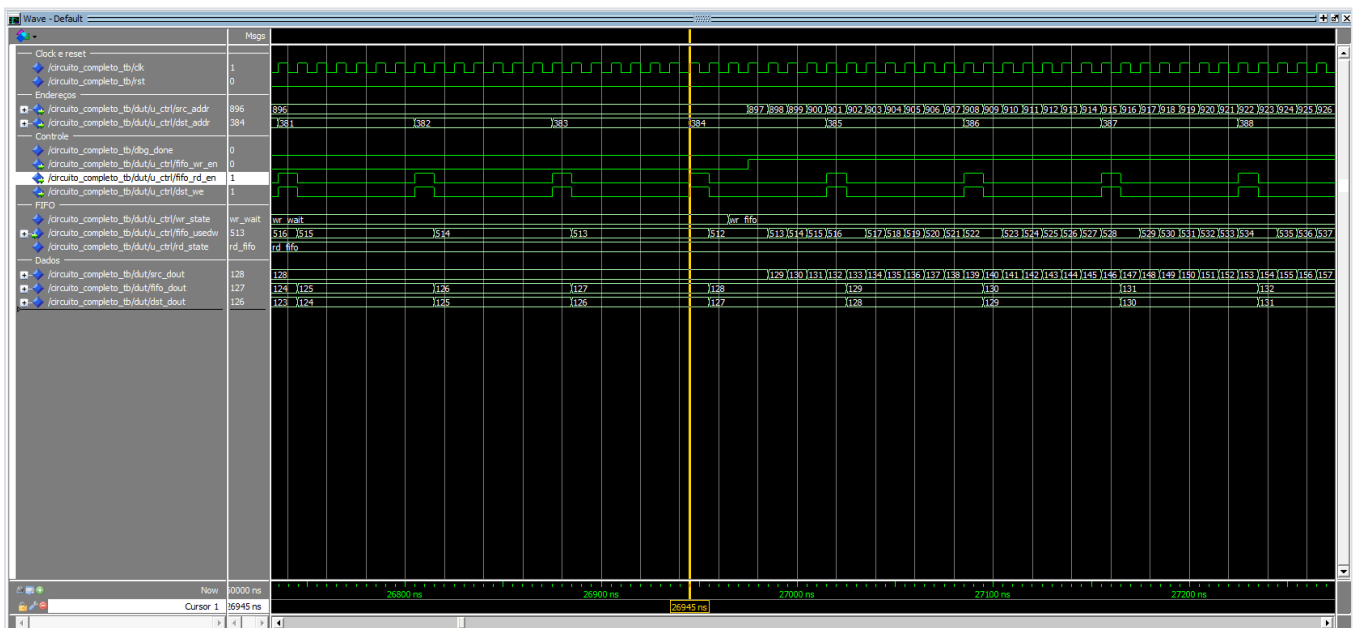
Aqui o circuito já finalizou a drenagem da FIFO e a máquina de leitura chegou a `rd_done`. O sinal `dbg_done` já indica término, o `usedw` está em zero e a BRAM de destino recebeu os valores até o último endereço. Essa é a imagem que comprova o fechamento completo do processo.

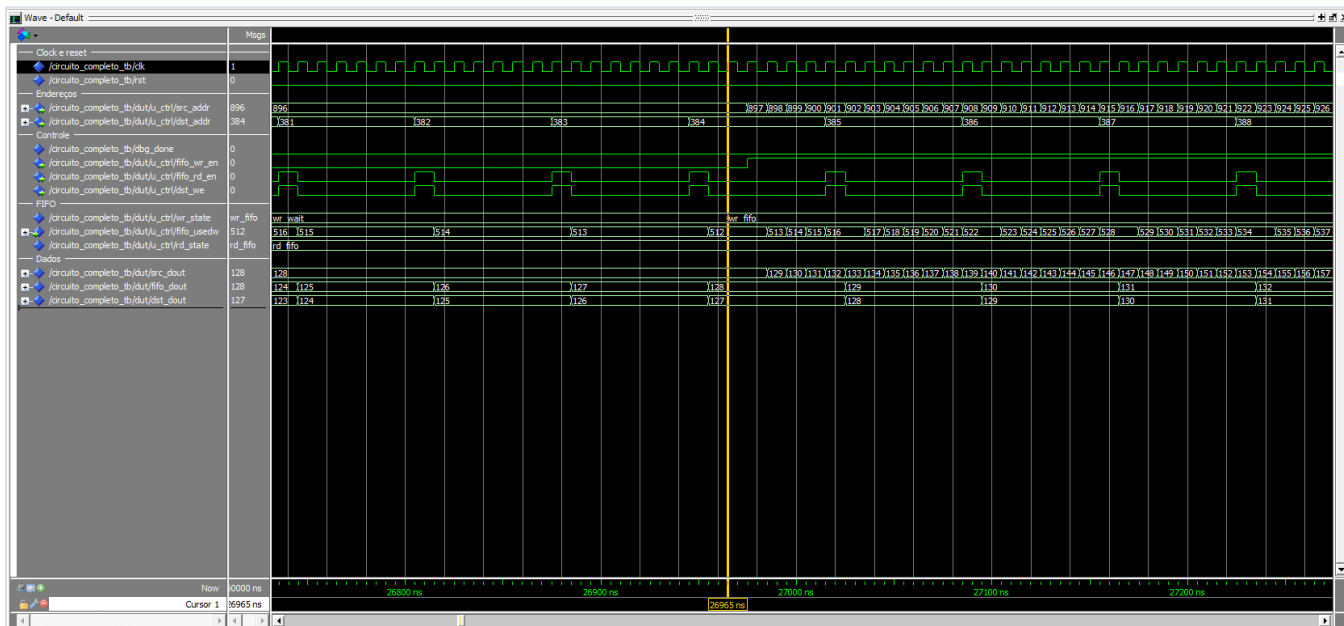
4.5 Limite superior da FIFO (75%)

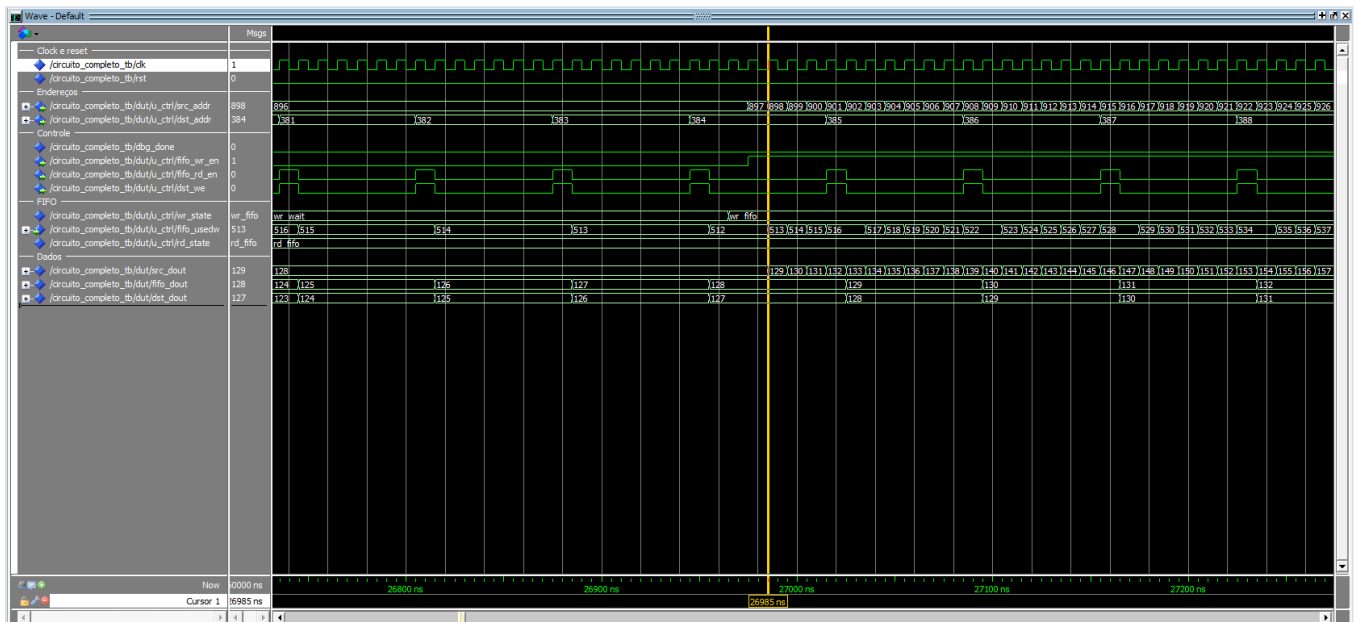
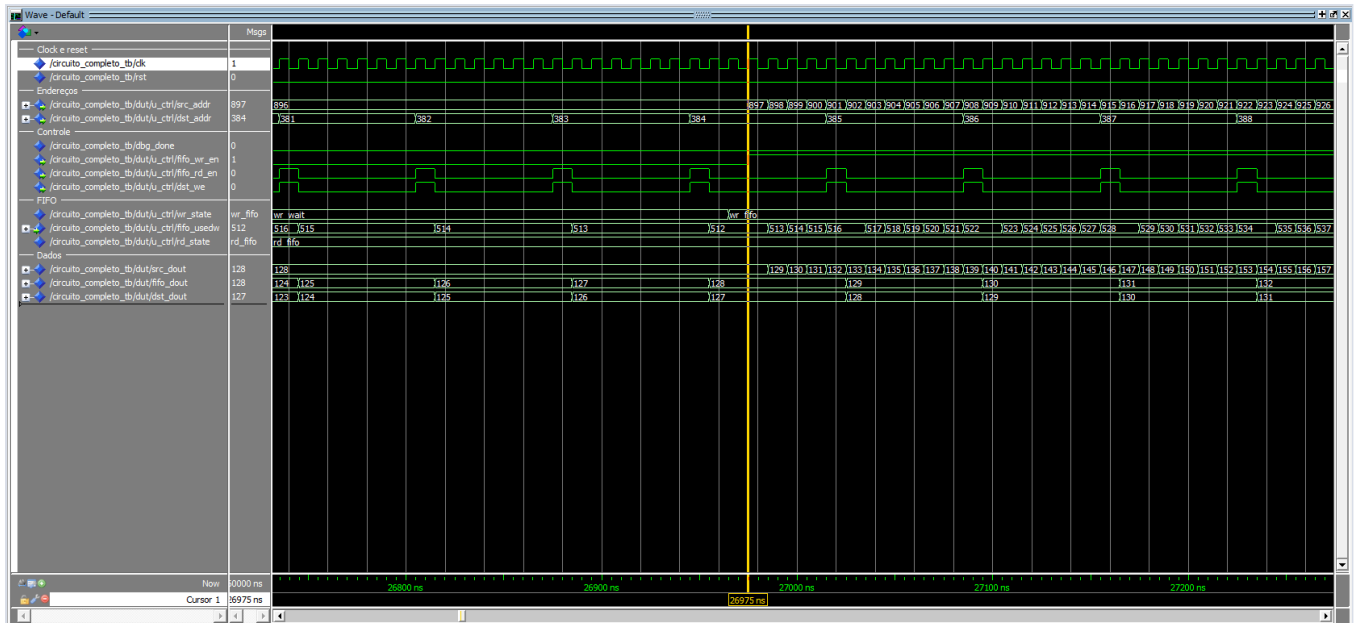


Esse zoom mostra a ocupação da FIFO crescendo até a faixa alta de controle, correspondente a 75% de uso da capacidade. Os valores de `usedw` sobem de forma contínua até a região em que o controlador precisa parar a escrita, e o sinal `wr_wait` aparece para impedir que a FIFO ultrapasse o limite superior.

4.6 Retomada da escrita (50%)







Nessas imagens é mostrada a primeira vez em que a FIFO chega a 50% de ocupação, que é o limiar inferior definido no enunciado. Foi necessário usar cinco imagens para mostrar o tempo em que `fifo_rd_en` e `dst_we` ficam ativos, permitindo que a FIFO seja drenada apenas uma vez a cada 7 clocks. Também fica visível que, só no clock seguinte após a FIFO atingir 50% (512), a máquina de escrita sai de `wr_wait` e volta para `wr_fifo`. No clock seguinte, `fifo_wr_en` volta a ficar ativo e, no clock seguinte desse, a ocupação da FIFO começa a subir novamente. O controle de fluxo fica bem visível nessa sequência, mostrando a escrita saindo de parada para ativa.

5. Conclusão

O sistema implementado atende aos requisitos da atividade, demonstrando corretamente:

- uso de BRAM gerada pelo Quartus como armazenamento;
- uso de FIFO gerada pelo Quartus para desacoplamento de taxas;

- controle de fluxo baseado em ocupa?o;
- opera?o com diferentes velocidades de leitura e escrita.

A valida?o por simula?o comprova o funcionamento correto tanto dos blocos individuais quanto do sistema integrado, evidenciando o comportamento esperado em todas as etapas da transfer?ncia de dados.